

3 Practice and Comprehensive Questions Solutions

```
# The following libraries are used, so please load them beforehand
import numpy as np
import numpy.random as random
import scipy as sp
import pandas as pd
from pandas import Series, DataFrame
import os
import zipfile
import requests
import io

# Visualization Library
import matplotlib.pyplot as plt
import matplotlib as mpl
import seaborn as sns
%matplotlib inline

# Displayed to 3 decimal places
%precision 3
```

```
'%.3f'
```

```
## Loading the students-mat data
# Specify url with data
url = 'http://archive.ics.uci.edu/ml/machine-learning-databases/00356/student.zip'

# Acquire data from url
r = requests.get(url, stream=True)

# Read and extract the zipfile
z = zipfile.ZipFile(io.BytesIO(r.content))
z.extractall()
```

<Practice Question 3-1>

For the following data table, let's output data where `city` is Tokyo or `birth_year` is greater than 1990.

```
# Prepare data1
data1 = {
    'id': ['0', '1', '2', '3', '4', '6', '8', '11', '12', '13'],
    'city': ['Tokyo', 'Osaka', 'Kyoto', 'Hokkaido', 'Tokyo', 'Tokyo', 'Osaka', 'Kyoto', 'Hokkaido', 'Tokyo'],
    'birth_year': [1990, 1989, 1992, 1997, 1982, 1991, 1988, 1990, 1995, 1981],
    'name': ['Hiroshi', 'Akiko', 'Yuki', 'Satoru', 'Steeve', 'Mituru', 'Aoi', 'Tarou', 'Suguru', 'Mitsuo']
}
df1 = DataFrame(data1)
df1
```

	id	city	birth_year	name	
0	0	Tokyo	1990	Hiroshi	
1	1	Osaka	1989	Akiko	
2	2	Kyoto	1992	Yuki	
3	3	Hokkaido	1997	Satoru	
4	4	Tokyo	1982	Steeve	
5	6	Tokyo	1991	Mituru	
6	8	Osaka	1988	Aoi	
7	11	Kyoto	1990	Tarou	
8	12	Hokkaido	1995	Suguru	
9	13	Tokyo	1981	Mitsuo	

Next steps: [Generate code with df1](#) [New interactive sheet](#)

```
df1[(df1['city']=='Tokyo') | (df1['birth_year']>1990)]
```

	id	city	birth_year	name
0	0	Tokyo	1990	Hiroshi
2	2	Kyoto	1992	Yuki
3	3	Hokkaido	1997	Satoru
4	4	Tokyo	1982	Steeve
5	6	Tokyo	1991	Mituru
8	12	Hokkaido	1995	Suguru
9	13	Tokyo	1981	Mitsuo

✓ <Practice Question 3-2>

Load the `student-mat.csv`. This dataset contains data of students' attributes and math scores in secondary education of two Portuguese schools.

Columns `G1` and `G2` represent the students' math grade in the first and second period, respectively. Create a new column named `G1_G2_diff`, to calculate the change in grades from the first to the second period. Then, sort the dataset in **descending** order.

```
# Load the data
student_data_math = pd.read_csv('student-mat.csv', sep=';')

student_data_math["G1_G2_diff"] = student_data_math["G2"] - student_data_math["G1"]
student_data_math.sort_values("G1_G2_diff", ascending=False)
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	freetime	goout	Dalc	Walc	health	absences
4	GP	F	16	U	GT3	T	3	3	other	other	...	3	2	1	2	5	4
96	GP	M	16	R	GT3	T	4	3	services	other	...	3	3	1	1	4	2
161	GP	M	15	R	GT3	T	3	2	other	other	...	4	4	1	4	3	6
254	GP	M	17	R	GT3	T	2	1	other	other	...	4	2	2	4	5	0
54	GP	F	15	U	LE3	A	3	3	other	other	...	3	4	4	4	1	6
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
131	GP	F	15	U	GT3	T	1	1	at_home	other	...	3	3	1	2	4	0
134	GP	M	15	R	GT3	T	3	4	at_home	teacher	...	3	3	1	1	5	0
136	GP	M	17	R	GT3	T	3	4	at_home	other	...	4	5	2	4	5	0
135	GP	F	15	U	GT3	T	4	4	services	at_home	...	3	3	1	1	5	0
130	GP	F	15	R	GT3	T	3	4	services	teacher	...	2	2	2	2	5	0

395 rows × 34 columns

✓ <Practice Question 3-3>

Let's do an internal join between the following two data tables with the same `id` value.

```
# Prepare data 4
data4 = {
    'id': ['0', '1', '2', '3', '4', '6', '8', '11', '12', '13'],
    'city': ['Tokyo', 'Osaka', 'Kyoto', 'Hokkaido', 'Tokyo', 'Tokyo', 'Osaka', 'Kyoto', 'Hokkaido', 'Tokyo'],
    'birth_year': [1990, 1989, 1992, 1997, 1982, 1991, 1988, 1990, 1995, 1981],
    'name': ['Hiroshi', 'Akiko', 'Yuki', 'Satoru', 'Steeve', 'Mituru', 'Aoi', 'Tarou', 'Suguru', 'Mitsuo']
}
df4 = DataFrame(data4)
print(df4)

# Prepare data 5
data5 = {
    'id': ['0', '1', '3', '6', '8'],
    'math': [20, 30, 50, 70, 90],
    'English': [30, 50, 50, 70, 20],
}
```

```
'sex':['M','F','F','M','M'],
'index_num':[0,1,2,3,4]
}
df5 = DataFrame(data5)
print(df5)
```

```
   id  city birth_year  name
0  0  Tokyo      1990 Hiroshi
1  1  Osaka      1989  Akiko
2  2  Kyoto      1992   Yuki
3  3  Hokkaido     1997  Satoru
4  4  Tokyo      1982  Steeve
5  6  Tokyo      1991  Mituru
6  8  Osaka      1988   Aoi
7 11  Kyoto      1990  Tarou
8 12  Hokkaido     1995  Suguru
9 13  Tokyo      1981 Mitsuo
   id  math  English sex  index_num
0  0    20     30    M         0
1  1    30     50    F         1
2  3    50     50    F         2
3  6    70     70    M         3
4  8    90     20    M         4
```

```
# Answer
pd.merge(df4, df5, on='id')
```

	id	city	birth_year	name	math	English	sex	index_num
0	0	Tokyo	1990	Hiroshi	20	30	M	0
1	1	Osaka	1989	Akiko	30	50	F	1
2	3	Hokkaido	1997	Satoru	50	50	F	2
3	6	Tokyo	1991	Mituru	70	70	M	3
4	8	Osaka	1988	Aoi	90	20	M	4

✓ <Practice Question 3-4>

Using the data from <Exercise 3-3>, let's vertically join the following data for `df4`.

```
# Preparing data
data6 = {
    'id':['70','80','90','120','150'],
    'city':['Chiba','Kanagawa','Tokyo','Fukuoka','Okinawa'],
    'birth_year':[1980,1999,1995,1994,1994],
    'name':['Suguru','Kouichi','Satochi','Yukie','Akari']
}
df6 = DataFrame(data6)
```

```
pd.concat([df4, df6])
```

	id	city	birth_year	name
0	0	Tokyo	1990	Hiroshi
1	1	Osaka	1989	Akiko
2	2	Kyoto	1992	Yuki
3	3	Hokkaido	1997	Satoru
4	4	Tokyo	1982	Steeve
5	6	Tokyo	1991	Mituru
6	8	Osaka	1988	Aoi
7	11	Kyoto	1990	Tarou
8	12	Hokkaido	1995	Suguru
9	13	Tokyo	1981	Mitsuo
0	70	Chiba	1980	Suguru
1	80	Kanagawa	1999	Kouichi
2	90	Tokyo	1995	Satochi
3	120	Fukuoka	1994	Yukie
4	150	Okinawa	1994	Akari

```
hier_df1 = DataFrame(
    np.arange(12).reshape((3,4)),
    index = [['c','d','d'], [1,2,1]],
    columns = [
        ['Kyoto','Nagoya','Hokkaido','Kyoto'],
        ['Yellow','Yellow','Red','Blue']
    ]
)

hier_df1.index.names = ['key1','key2']
hier_df1.columns.names = ['city','color']
hier_df1
```

	city	Kyoto	Nagoya	Hokkaido	Kyoto
	color	Yellow	Yellow	Red	Blue
key1	key2				
c	1	0	1	2	3
d	2	4	5	6	7
	1	8	9	10	11

Next steps: [Generate code with hier_df1](#) [New interactive sheet](#)

▼ <Practice Question 3-5>

Load the `student-mat.csv`, the dataset with students' attributes and math scores in secondary education of two Portuguese schools. Column `absences` represent the number of absences. Let's count the number of people in each of the following three bins in this column. Note that the default behavior of `cut` is a closed interval on the right side. This time, specify the option `right=False` for the `cut` function to make the right side the open interval.

```
# Granularity of division
absences_bins = [0, 1, 5, 100]
```

```
# Answer
student_data_math = pd.read_csv('student-mat.csv', sep=';')

student_data_math_ab_cut_data = pd.cut(student_data_math.absences, absences_bins, right=False)
pd.value_counts(student_data_math_ab_cut_data)
```

```
/tmp/ipykernel_455/2467858326.py:5: FutureWarning: pandas.value_counts is deprecated and will be removed in a future version. Use
pd.value_counts(student_data_math_ab_cut_data)
```

```
count
absences
[5, 100)    151
[1, 5)      129
[0, 1)      115

dtype: int64
```

✓ <Practice Question 3-6>

With the same data as above, let's divide the “absences” column into three bins using the `qcut` function.

```
# Answer
student_data_math_ab_qcut_data = pd.qcut(student_data_math.absences,3)
pd.value_counts(student_data_math_ab_qcut_data)
```

```
/tmp/ipykernel_455/957399414.py:3: FutureWarning: pandas.value_counts is deprecated and will be removed in a future version. Use
pd.value_counts(student_data_math_ab_qcut_data)
```

```
count
absences
(-0.001, 2.0]    183
(6.0, 75.0]     115
(2.0, 6.0]       97

dtype: int64
```

✓ <Practice Question 3-7>

Let's use `student-mat.csv` to do the Pandas aggregation process.

First, let's find the average score for each `G1` on the basis of school (`school`). Columns `G1` and `school` represent the students' math grade in the first period and the students' school, respectively.

```
student_data_math.groupby(['school'])['G1'].mean()
```

```
G1
school
GP    10.939828
MS    10.673913

dtype: float64
```

✓ <Practice Question 3-8>

Columns `G2` and `G3` represent the students' math grade in the second period and final grade, respectively. Column `address` represent where they live ('U' - urban or 'R' - rural).

Let's use “student-mat.csv” to find the average score for columns `G1`, `G2`, and `G3`, respectively, grouped by school (`school`) and address (`address`).

```
# Answer
student_data_math.groupby(['school', 'address'])[['G1', 'G2', 'G3']].mean()
```

		G1	G2	G3	
school	address				
GP	R	10.682540	10.031746	9.761905	
	U	10.996503	10.947552	10.650350	
MS	R	9.960000	9.320000	8.880000	
	U	11.523810	11.238095	11.000000	

The reason why the results are different from those in <Exercise 3-10> above is that the answer to <Exercise 3-10> is a Series type and this time the answer is a DataFrame.

✓ <Practice Question 3-9>

Let's use "student-mat.csv" to calculate the maximum and minimum values of G1, G2, and G3 together, based on school (school) and address (address).

```
# Answer
functions = ['max', 'min']
student_data_math2 = student_data_math.groupby(['school', 'address'])
student_data_math2[['G1', 'G2', 'G3']].agg(functions)
```

		G1		G2		G3		
		max	min	max	min	max	min	
school	address							
GP	R	18	3	18	0	18	0	
	U	19	4	19	0	20	0	
MS	R	19	6	18	5	19	0	
	U	18	6	16	5	16	0	

✓ <Practice Question 3-10>

For the following data, if there is even one column of NaN, delete it and display the result.

```
##@title
# Prepare data
import numpy as np
from numpy import nan as NA
import pandas as pd

df2 = pd.DataFrame(np.random.rand(15,6))

# Make it NA
df2.iloc[2,0] = NA
df2.iloc[5:8,2] = NA
df2.iloc[7:9,3] = NA
df2.iloc[10,5] = NA

df2
```

	0	1	2	3	4	5
0	0.202672	0.495871	0.498808	0.584196	0.433613	0.493051
1	0.266455	0.143381	0.868184	0.289630	0.325853	0.577430
2	NaN	0.449522	0.939223	0.800417	0.889775	0.252236
3	0.039590	0.984056	0.584648	0.805368	0.430843	0.158155
4	0.345213	0.481179	0.890208	0.951061	0.597859	0.406284
5	0.158468	0.365767	NaN	0.275428	0.719585	0.019773
6	0.402816	0.721078	NaN	0.561813	0.312110	0.302284
7	0.076715	0.945503	NaN	NaN	0.424978	0.201109
8	0.600020	0.935091	0.940364	NaN	0.496138	0.871791
9	0.981018	0.376083	0.926281	0.487461	0.675523	0.935118
10	0.311044	0.200009	0.810637	0.814500	0.419094	NaN
11	0.905940	0.197402	0.508470	0.328920	0.153069	0.330047
12	0.720967	0.434319	0.804551	0.786066	0.037614	0.097108
13	0.220010	0.091230	0.892230	0.123674	0.880033	0.741754
14	0.824381	0.637660	0.493490	0.178559	0.229839	0.384107

Next steps: [Generate code with df2](#) [New interactive sheet](#)

```
# Answer
df2.dropna()
```

	0	1	2	3	4	5
0	0.202672	0.495871	0.498808	0.584196	0.433613	0.493051
1	0.266455	0.143381	0.868184	0.289630	0.325853	0.577430
3	0.039590	0.984056	0.584648	0.805368	0.430843	0.158155
4	0.345213	0.481179	0.890208	0.951061	0.597859	0.406284
9	0.981018	0.376083	0.926281	0.487461	0.675523	0.935118
11	0.905940	0.197402	0.508470	0.328920	0.153069	0.330047
12	0.720967	0.434319	0.804551	0.786066	0.037614	0.097108
13	0.220010	0.091230	0.892230	0.123674	0.880033	0.741754
14	0.824381	0.637660	0.493490	0.178559	0.229839	0.384107

<Practice Question 3-11>

Fill in the `NaN` with zeros for the data prepared in <Exercise 3-13>.

```
# Answer
df2.fillna(0)
```

	0	1	2	3	4	5
0	0.202672	0.495871	0.498808	0.584196	0.433613	0.493051
1	0.266455	0.143381	0.868184	0.289630	0.325853	0.577430
2	0.000000	0.449522	0.939223	0.800417	0.889775	0.252236
3	0.039590	0.984056	0.584648	0.805368	0.430843	0.158155
4	0.345213	0.481179	0.890208	0.951061	0.597859	0.406284
5	0.158468	0.365767	0.000000	0.275428	0.719585	0.019773
6	0.402816	0.721078	0.000000	0.561813	0.312110	0.302284
7	0.076715	0.945503	0.000000	0.000000	0.424978	0.201109
8	0.600020	0.935091	0.940364	0.000000	0.496138	0.871791
9	0.981018	0.376083	0.926281	0.487461	0.675523	0.935118
10	0.311044	0.200009	0.810637	0.814500	0.419094	0.000000
11	0.905940	0.197402	0.508470	0.328920	0.153069	0.330047
12	0.720967	0.434319	0.804551	0.786066	0.037614	0.097108
13	0.220010	0.091230	0.892230	0.123674	0.880033	0.741754
14	0.824381	0.637660	0.493490	0.178559	0.229839	0.384107

<Practice Question 3-12>

For the data prepared in <Exercise 3-13>, fill in the `NaN` with the average value of each column.

```
# Answer
df2.fillna(df2.mean())
```

	0	1	2	3	4	5
0	0.202672	0.495871	0.498808	0.584196	0.433613	0.493051
1	0.266455	0.143381	0.868184	0.289630	0.325853	0.577430
2	0.432522	0.449522	0.939223	0.800417	0.889775	0.252236
3	0.039590	0.984056	0.584648	0.805368	0.430843	0.158155
4	0.345213	0.481179	0.890208	0.951061	0.597859	0.406284
5	0.158468	0.365767	0.763091	0.275428	0.719585	0.019773
6	0.402816	0.721078	0.763091	0.561813	0.312110	0.302284
7	0.076715	0.945503	0.763091	0.537469	0.424978	0.201109
8	0.600020	0.935091	0.940364	0.537469	0.496138	0.871791
9	0.981018	0.376083	0.926281	0.487461	0.675523	0.935118
10	0.311044	0.200009	0.810637	0.814500	0.419094	0.412160
11	0.905940	0.197402	0.508470	0.328920	0.153069	0.330047
12	0.720967	0.434319	0.804551	0.786066	0.037614	0.097108
13	0.220010	0.091230	0.892230	0.123674	0.880033	0.741754
14	0.824381	0.637660	0.493490	0.178559	0.229839	0.384107

```
# For checking
df2.mean()
```

```
0
0 0.432522
1 0.497210
```

3.7 Comprehensive Question

```
2 0.763091
3 0.537469
```

Comprehensive Question 3-1 Data Manipulation

```
4 0.468395
5 0.412160
```

Using the data 'student-mat.csv', which is the data showing math scores, answer the following questions.

- (1) For the above data, calculate the average score of `G1` by age (`age`) x sex (`sex`) and create a table (`table`) such that the vertical axis is age (`age`) and the horizontal axis is sex (`sex`).
- (2) For the result table displayed in (1), delete all the rows (records) that are NaN.

```
# Answer
# (1)
student_data_math.groupby(['age', 'sex'])['G1'].mean().unstack()
```

sex	F	M
age		
15	10.052632	12.250000
16	10.203704	11.740000
17	11.103448	10.600000
18	10.883721	10.538462
19	10.642857	9.700000
20	15.000000	13.000000
21	NaN	10.000000
22	NaN	6.000000

```
# Answer
# (2)
student_data_math.groupby(['age', 'sex'])['G1'].mean().unstack().dropna()
```

sex	F	M
-----	---	---